

1. (계획 수립 단계인 경우) Physical Computing 혹은 순수 SW작품 연구(개발) 계획서

I 감귤의 색상과 당도 분석 선과기 연구 개발 주제

1. 개발 동기 및 필요성

지구온난화로 인해 내륙지역의 감귤 재배 면적과 감귤 생산량이 늘고 있다. 제주도는 섬이라는 특성 때문에 감귤이 내륙으로 들어가기 위해서는 배나 비행기를 방법밖엔 없다. 내륙보다 더 배송시간이 길다는 특성 때문에 제주도의 감귤은 부패, 흠집에 더 신경쓰지 않으면 상품성 낮은 귤이 소비자에게 도착하게 된다.

감귤의 품질은 일반적으로 당도와 산도로 결정된다. 특히, 당도는 감귤의 맛을 결정하기 때문에 매우 중요한 요소이다. 현재 농가에서 가장 많이 사용하는 감귤 당도 측정 방법은 휴대용 착즙당도계 및 비파괴당도계로 측정하는 방식이다. 이 방법은 개인이 간단하게 당도를 측정할 수 있는 방법이지만, 착즙당도계는 감귤을 파괴한 뒤 직접 당도를 측정하기 때문에 측정한 귤은 상품으로 쓸 수 없으며, 비파괴당도계는 가격이 많이 나가고, 무거워서 휴대하기 어렵다는 단점이 있다.

2. 유사 연구

최근 연구들에 따르면 다른 과일들은 껍질의 모양이나 외관으로도 품질을 평가하는 방법들이 연구되고 있다. 사과와 경우 꽃받침 형상이나 사과의 껍질 색깔을 이용하여 당도를 측정하는 방법도 나왔다. 하지만, 유독 귤의 품질을 판단하는 방법이 적다. 그래서 우리는 새로운 감귤의 품질 측정 방식과 소형농가에도 놓기 편한 AI 기술을 적용한 감귤의 색상과 당도 분석 선과기제작하고자 한다.

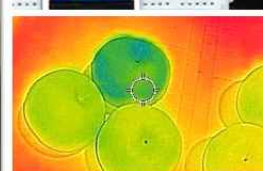
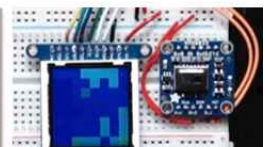
II 감귤의 색상과 당도 분석 선과기 제작

1. 작품 설계

- 목표 : 라즈베리파이와 아두이노를 이용한 감귤 선과 장치를 이용해 감귤의 사진을 찍어 YOLO를 이용하여 감귤 사진을 검출하고 감귤에 가장 많이 가지고 있는 3가지의 색을 검출해 감귤의 껍질 색을 이용한 감귤 당도 예측 모델[random forest 모델]을 이용해 감귤의 당도를 예측하고 크기별 당도별 감귤을 분류

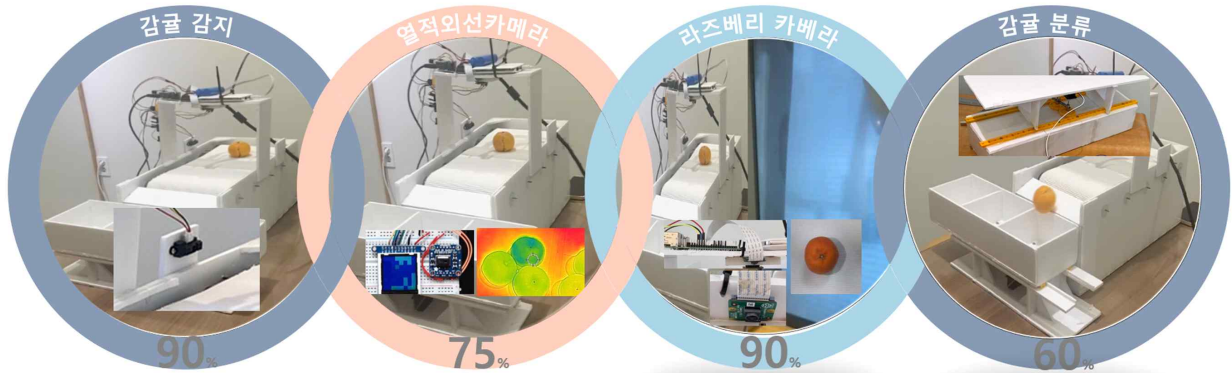
[1] 감귤 선과기

- 감귤 선과기에 설치된 주요 센서 및 동작 제어 회로와 각각의 기능
 - 360도 서보모터 : 원기둥을 모터가 회전시켜 원기둥을 감싸고 있는 컨베이어 벨트가 돌아감, 서보모터가 판을 회전시키면 판과 연결되어있는 선과기가 같이 회전하게됨
 - 적외선 센서 : 귤과의 거리가 15cm를 기준으로 먼지 가까운지 측정
 - ※ 귤이 없을 때 거리 값을 일정하게 측정되도록 하기 위해 적외선 센서 반대에 크기가 비슷한 판을 붙임
 - 라즈베리파이 카메라 : 귤이 지나가는 컨베이어벨트 위에 설치되어 귤의 윗모습을 찍음
 - 열적외선 카메라 : 64픽셀의 값으로 감귤의 내부 온도 측정



라즈베리파이 카메라와 열적외선 카메라

- 감귤 당도 예측 모델을 이용한 감귤 선과기 동작 실험



감귤 감지

적외선 센서를 이용

컨베이어벨트 위에서 이동하는 감귤을 적외선 센서를 이용하여 감지

열적외선카메라

당도 및 불량 분류

열적외선카메라를 이용하여 과일의 내부 온도를 측정하여 당도 예측 학습 데이터로 사용, 과일의 상처, 상한 과일 분류에 사용

라즈베리카메라

당도 및 불량 분류

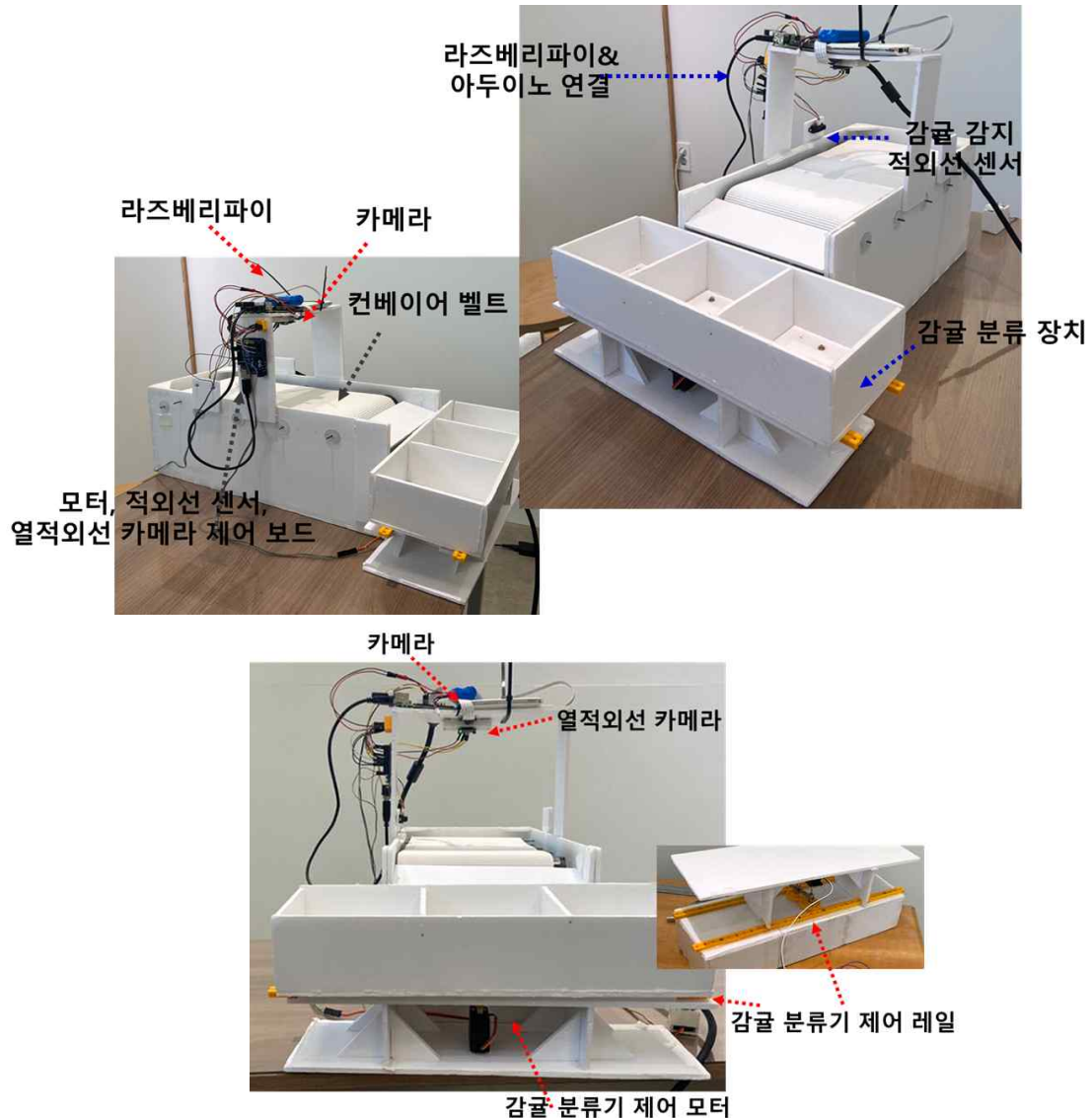
카메라를 이용하여 과일의 겉질을 측정하여 겉질의 색을 추출하여 감귤의 당도 예측 학습 데이터로 사용

감귤 분류 장치

레일과 모터를 이용

감귤의 분류를 당도 높은 감귤, 당도가 낮은 감귤, 불량인 감귤 3가지로 분류하여 전달받은 값에 따라 모터와 레일을 이용해 분류

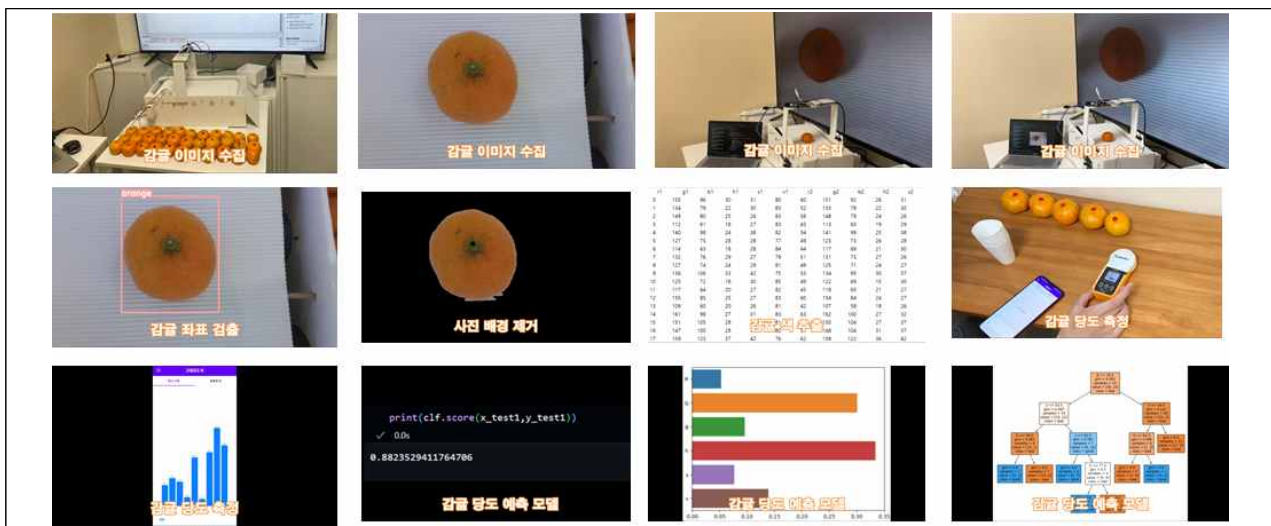
- 감귤 선과기 구조



[2] 감귤 당도 예측 모델

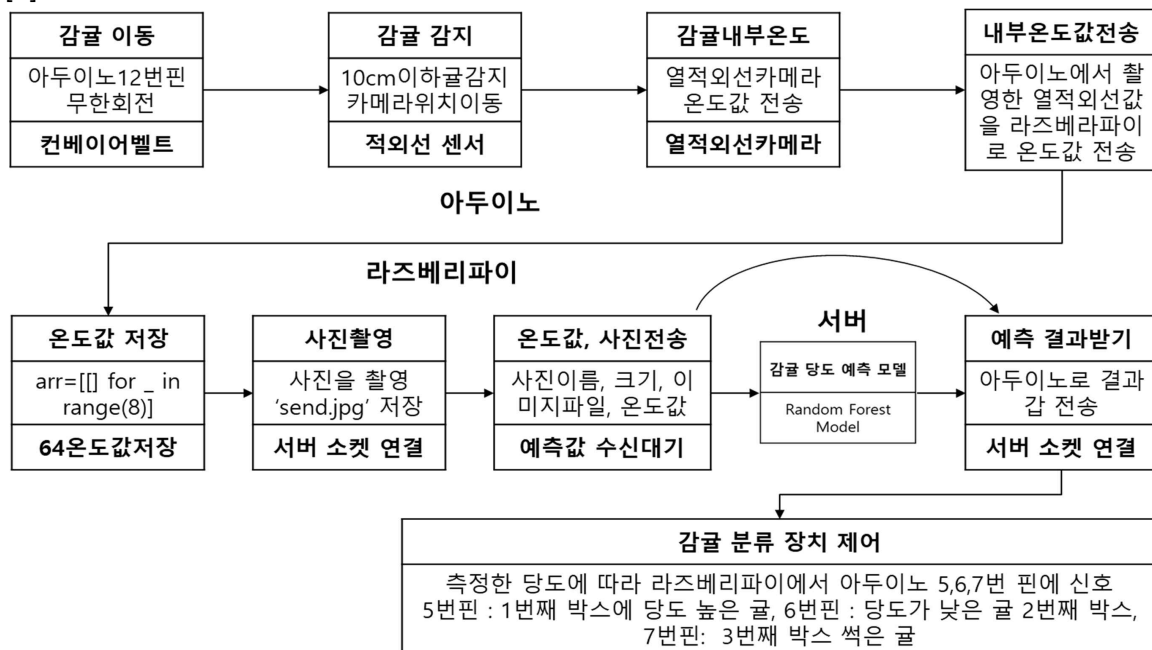
감귤 당도 예측 모델 학습 과정	감귤 당도 예측 과정
- 데이터 생성 1) 감귤의 좌표 추출-YOLO 모델 2) 사진의 배경 제거-grabcut 알고리즘 3) 감귤의 색 추출-Python Image Library 4) 감귤 내부 온도 측정 5) 재배 지역의 기후 6) 감귤의 당도 측정 - 당도 예측 모델 학습-random forest 모델	1. 감귤이 있는지 확인-YOLO 모델 2. 감귤의 좌표 추출-YOLO 모델 3. 사진의 배경 제거-grabcut 알고리즘 4. 감귤의 색 추출-Python Image Library 5. 감귤의 당도 예측-random forest 모델

• 감귤 당도 예측 모델 학습 단계



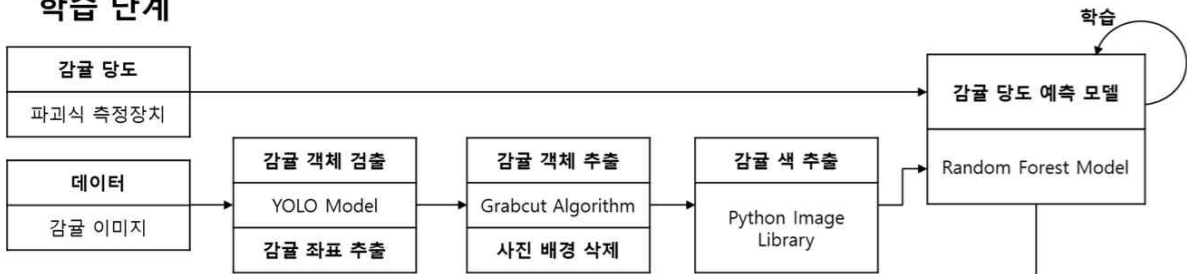
- 작품 설계

[1] 감귤 선과기 동작에 따른 핵심 소스코드



[2] 감귤 당도 예측 모델 학습 및 예측 단계 흐름도

학습 단계



예측 단계



[3] 작품 동작 과정

작품 동작 과정

1. 감귤을 컨베이어 벨트에 올려놓는다.
2. 적외선 센서가 감귤을 감지하면 컨베이어벨트가 2초 후 멈춘다.

3. 아두이노 열적외선 카메라가 동작한 후 값을 라즈베리파이로 보낸다.
4. 라즈베리파이에 연결된 카메라가 감귤의 사진을 찍은 후 서버 컴퓨터로 파일을 전송한다.

5. 서버는 학습 모델을 이용하여 감귤의 당도가 높은줄(1), 당도가 낮은줄(0), 불량줄(-1) 3개 중 하나의 값을 감귤 선과기로 보낸다.
6. 이 신호를 받으면 컨베이어벨트가 다시 움직인다.
7. 감귤 분류 장치는 1, 0, -1의 값에 맞게 움직여 껍을 분류통에 담는다.



2. 작품 제작 일지

[1] 작품 제작 방향에 대한 자료 조사[담당자 : 조왕현]

2월 1일 : 과일 당도 측정 방법

2월 8일 : 감귤 당도 측정관련 자료 조사

과일의 당도를 표시하는 단위는 브릭스(Brix)라고 하는데, 브릭스란 과일중 100g에 녹아 있는 당분의 그램(gram)을 말한다. 첫 번째는 감귤농장의 농업기술센터에서 껍질,착,종자,껍질,종자,껍질로 측정하는 방식이다. 이 방법은 가장 정확한 측정 방법이지만, 파괴식 측정 방식이며 농가에 서는 농장기술센터로 매년 직접 방문해야 하는 번거로움이 있다. 번째는 휴대용 측정장치 및 비파괴당도계로 측정하는 방식이다. 이 방법은 개인이 손쉽게 측정 가능한 방법이지만, 감귤농장 정식 기계보다 당도 수치의 정확성이 떨어져 특히 0.5 Brix(1 Brix는 물 100g에 당분이 1g 녹아 있는 것을 의미) 이상 오차 차 이가 발생해 현장에서 사용하기에는 아직 많이 부족하다. 또한, 두 방법 모두 현재 시점의 측정치에 예측 측정이 아니다.

<https://m.cafe.daum.net/mispectra/741M/9>

비파괴 선행기원리
Spectra Science Co., Ltd. 비파괴 분석법인 농산물외 적외선, 라만, NIR, VIS, UV, X-ray, etc.

비파괴 분석법인 농산물의 외부에 광, 진동, 전자기 등의 물리적 에너지를 주고 혹은 반사시키고 이때 생기는 에너지 변화를 해석하여 시료의 화학적 특성이나 성분을 분석하는 간접적인 물질 측정 방법이다. 비파괴 분석법은 분해, 추출, 정제, 농축, 가열 등 시료의 전체를 필요로 하는 종래의 화학 분석에 비해 다량의 화학 약품이나 숙련된 기술자를 필요로 하지 않고 종말 시료를 반복하여 사용할 수 있을 뿐 아니라 다양한 물질, 성분을 동시에 분석할 수 있다.

같은 야간촬영용 적외선카메라에 쓰이는 '근적외선'을 쬔어 당도를 알아냅니다. 곧 속으로 들어간 근적외선은 당 성분과 만나면 여러 각도로 쬔어. 이렇게 곧 밖으로 나온 근적외선의 세기가 얼마나 변했는지를 측정해 당도를 알아내는 거죠. 가시광선 등 근적외선 외의 다른 빛은 껍질 통과해도 큰 변화가 없어 측정이 거의 불가능하다고 합니다. 껍질 통과한 근적외선의 세기 변화는 당도 단위인 '브릭스(Brix)'로 환산할 수 있습니다. 1브릭스는 물 100g에 당분이 1g 녹아있다는 뜻이에요.

<https://blog.naver.com/dand293/222566291662>

맛있는 과일을 알아볼 수 있는 당도측정기 원리
맛있는 과일을 고를 수 있는 가장 간단한 방법은 직접 확인하는 것이 아닐까?
blog.naver.com

2월 15일 : 당도 측정기 자료 조사

Much better 디지털당도측정기 (281,800원)

해아림 H-100C 감귤류패밀러형 비파괴 당도계 (3,300,000원)

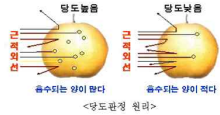
굴절당도계(파괴식 당도계)의 원리

굴절 당도계는 빛의 굴절 변화를 이용하여 액체의 당분 변화에 따라 굴절 변화를 전기적 또는 광학적 신호로 나타낸 것이다. 물질에 따라 굴절률이 다른데 농도가 진한 액일수록 더욱더 많이 구부러져 보인다. 이 원리를 이용하여 물질의 진한 정도를 측정하는 것이다.

	파괴식 당도계	비파괴식 당도계
장점	비파괴식 당도계에 비해 당도 값에 오차가 적어 더 정밀한 당도 값을 알아낼 수 있음	파괴식 당도계에 비해 표면에 대가만 해도 당도를 측정할 수 있어 편리하고 빠르게 사용할 수 있음 식품 그대로를 파괴하지 않고 측정하여 효율적임
단점	과일에서 즙을 추출해야 하므로 과일의 품질이 떨어짐	파괴식 당도계에 비해 가격이 많이 나가고 정밀한 값을 알아내기 힘들

비파괴식 당도계의 원리

과일을 야간 촬영용 적외선카메라에 쓰이는 근적외선을 쬔어 당도를 알아낸다. 굴 속으로 들어간 근적외선은 당 성분과 만나면 여러 각도로 꺾이는데 이렇게 굴 밖으로 나온 근적외선의 세기가 얼마나 변했는지를 측정해 당도를 알아내는 것이다. 근적외선 외의 다른 빛은 과일을 통과해도 큰 변화가 없어 측정이 거의 불가능하다.



2월 22일 : 분광광도법이란?

- 분광광도법과 분광광도계의 원리
- 비어-람베르트 법칙의 사용

1. 분광광도법

- 분광광도법이란 빛을 이용하여 물질의 정보를 알아내는 분석법이다.

2. 분광광도계

- 분광광도계의 구조

1.2. 분광광도계의 구조

빛이 시료를 통과하면서 일부가 시료에 흡수된다. 흡수되지 않고 통과한 나머지 빛은 검출기에 도달한다. 만약, 시료가 빛을 흡수했다면 검출기에 도달한 빛의 세기(P)는 처음 쏘준 빛의 세기(P₀)보다 약할 것이다. 장치는 처음 쏘준 빛의 세기(P₀)와 검출기에 도달한 빛의 세기(P)를 비교하고 시료가 얼마나 빛을 흡수했는지 알려준다.

장치는 우리에게 투광도(T) 형태로 정보를 준다. 투광도는 '처음 빛의 세기(P₀)'와 '검출된 빛의 세기(P)'의 비율이다.

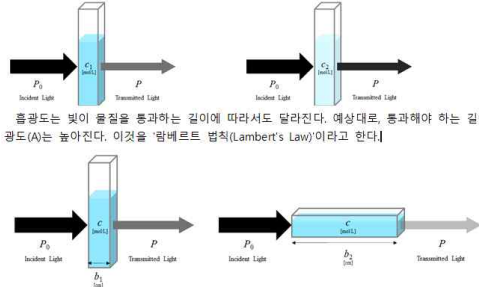
$$\text{투광도}(T) = \frac{P}{P_0}$$

시료가 빛을 흡수한 정도를 나타내는 흡광도(A)는 다음과 같이 정의된다.

$$\text{흡광도}(A) = -\log\left(\frac{P}{P_0}\right) = -\log T$$

3. 비어-람베르트 법칙

- 흡광도는 분석 과정에서 매우 매우 중요하다. 왜냐하면, 시료 내 빛을 흡수하는 화학종의 몰농도(c)에 따라 흡광도(A)가 달라지기 때문이다. 흡광도 A와 시료의 몰농도 c 사이 관계를 나타내는 법칙이 바로 '비어 법칙(Beer's Law)'이다.



3월 15일 : 비파괴 선별기와 과일 분류 인공지능

비파괴 선별기

과일에 투과 또는 반사된 근적외선이 분광광도계를 거쳐 생성된 스펙트럼을 통해 당도, 산도 등의 내부 품질을 판정한다

- 당도와 관련있는 근적외선 파장이 많이 흡수될수록 당도가 높다고 판단한다
- 반사식과 투과식이 있으며 반사식은 기존의 선별기에 그대로 적용할 수 있어 소형 저가의 비파괴 선별기를 구축할 수 있다

<https://m.blog.naver.com/gjb6560/801248615>

	반사식	투과식	반투과식
측정대상	사과, 배, 복숭아	감귤, 사과, 배, 포도	사과, 배, 복숭아, 멜론, 수박, 참외
측정인자	당도, 산도	당도, 산도, 숙도, 내부결함	당도, 숙도, 내부결함
시스템 구성			
측정부품	저대역폭, 광 조사부	저대역폭, 외부, 과일판지	

인공지능 과일 선별기

10개의 렌즈에서 근적외선을 쏘아 데이터를 얻은 후 이를 분석한다

-기존의 비파괴 측정기로 측정하기 어려운 내부 결함을 측정할 수 있다

-설정 범위에 따라 정확하게 과일을 선별하며 작업 속도도 매우 빠르다

<http://kenews.co.kr/mobile/article.html?no=82079>

감귤 당도 예측 인공지능

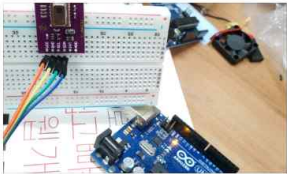
온도, 습도, 강수량 등의 기상 데이터를 기반으로 감귤의 당도를 예측한다

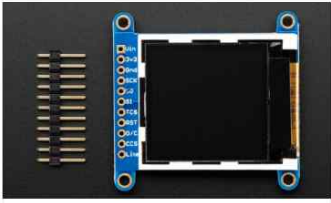
- 0.5brix 이상의 오차가 발생하는 휴대용 착즙 당도계, 비파괴 당도계와 달리 오차가 0.1~0.2로 0.5brix 이하로 나타난다

<https://koreascience.kr/article/JAKO202119759301791.pdf>

4월 19일 : 감귤의 내부를 촬영할 수 있는 카메라에 대한 자료조사

- 라즈베리 카메라를 이용해 감귤의 외부는 촬영할 수 있다.
- 하지만 자료조사나 논문 등을 보면 과일의 당도 측정에 대한 연구들에 근적외선 카메라를 사용한다는 내용들이 있었다.
- 그래서 우리가 사용할 수 있는 근적외선 카메라가 있는지 찾아봤지만 근적외선 카메라를 찾을 수는 없었다.
- 열적외선 카메라를 적용해보는 것은 어떨까해서 조사하고 테스트까지 해봤다.

<https://www.adafruit.com/product/2538>
 Adafruit AMG8833 IR Thermal Camera Breakout
 Add heat-vision to your project with an Adafruit AMG8833 8x8
www.adafruit.com
<https://blog.naver.com/roboholic84/221280979435>

 AMG8833 아두이노 열 화상 카메라 사용법 알아보기
 안녕하세요? 레거니움입니다. 이번에는 아두이노 열 화상 카메라인 A-
blog.naver.com

<https://www.devicemart.co.kr/spoods/view?no=1247904>

 Adafruit 1.44" Color TFT LCD Display with MicroSD Car...
 1.44 inch diagonal LCD TFT display / 128x128 resolution, 18-bit
www.devicemart.co.kr
☐ gpiotest.ino

[2] 감귤 선과기 제작에 필요한 자료조사 및 문제 해결 과정[담당자 : 김탁건]

2월 15일 : 컨베이어 벨트 제작
 • 컨베이어 벨트 설계 및 제작

제작 과정

1. 일면의 지름을 4cm이고 높이가 20cm인 원기둥 6개를 만들고 원기둥 중심을 25cm 떨어진 위치에 뚫는다.



2. 제작한 원기둥을 일정한 간격을 두고 배치하여 제작해 상자(가로 23cm 세로 49cm 높이 17cm)와 연결한다.



3. 원기둥들을 골판지로 감싼다.



문제 상황	길이가 안 맞으면 잘 끼워지지 않았다. 특히나 원기둥을 감싸는 골판지의 길이를 정확히 맞추었다.
-------	---

2월 22일 : 컨베이어 벨트 제작

작업 요약

컨베이어 벨트 움직임 제어

세부 내용

제작 과정

1. 서보모터를 마지막 원기둥과 연결해주고 모터를 돌릴 때 모터가 같이 돌지 않도록 서보모터를 고정한다.



2. 서보모터가 무한으로 돌도록 아두이노 코드를 입력한다.

```
#include <Servo.h>
Servo servo;
void setup() {
  // put your setup code here, to run once:
  servo.attach(9);
  servo.write(0);
}

void loop() {
  // put your main code here, to run repeatedly:
  servo.write(120);
  delay(1000);
}
```

3. 입력한 코드를 통해 컨베이어 벨트를 작동시킨다.
<http://blog.naver.com/takgun0831/222962678184>

문제 상황	벨트가 돌면서 옆면에 부딪히면서 돌아가다가 중간에 멈추는 상황이 생긴다.
-------	--

3월 1일 : 컨베이어 벨트 회전시키기

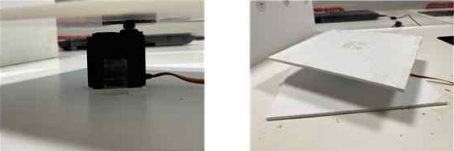
작업 요약

컨베이어 벨트가 회전하여 원하는 위치에 물건을 옮길 수 있도록 함.

세부 내용

제작 과정

1. 기계를 회전시키기 위해 서보모터를 판과 판 사이에 고정한다.



2. 한쪽 면은 지면에, 다른 한쪽 면은 컨베이어 벨트와 연결한다.

3. 간단한 서보모터 제어코드를 통해 작동하는지 확인한다.

```
#include <Servo.h>
Servo servo;
void setup() {
  // put your setup code here, to run once:
  servo.attach(9);
}

void loop() {
  // put your main code here, to run repeatedly:
  for(int i=90;i<180;i++){
    servo.write(i);
    delay(10);
  }
  delay(3000);
  for(int i=180;i>90;i--){
    servo.write(i);
    delay(10);
  }
  delay(3000);
}
```

문제 상황	처음 시도할때는 힘이 약한 서보모터를 써서 컨베이어 벨트의 무게 때문에 돌지 않았지만 다른 서보모터로 바꾸고 나서 잘 돌게 되었다.
-------	---

3월 8일 : 컨베이어 벨트 회전시키기

작업 요약	
적외선 센서가 특정 거리 값을 측정했을 때 컨베이어 벨트가 멈추도록 제어함	
세부 내용	
제작과정 1. 적외선 센서가 물체를 감지할 수 있도록 옆면보다 조금 높은 곳에다가 달아준다. 	
2. 적외선 센서가 측정하는 값을 시리얼 모니터에 출력하여 물체가 없을 때 와 있을 때 값을 확인한다. <pre> #include <Servo.h> Servo servo; int distance=0; void setup() { // put your setup code here, to run once: Serial.begin(9600); servo.attach(9); } void loop() { // put your main code here, to run repeatedly: int volt=map(analogRead(A0), 0,1023,0,5000); distance=(27.61/(volt-0.1696))*1000; Serial.print(distance); Serial.println("cm"); } </pre> 3. 물체가 있을 때 거리 값이 측정되면 컨베이어 벨트가 멈추도록 한다. https://blog.naver.com/takgun0831/222982678184	
문제 상황	적외선 센서가 미세한 움직임도 감지하다 보니 적외선 센서 반대쪽 벽을 놓여야했다.

3월 15일 : 컨베이어 벨트 회전시키기 및 문제 해결

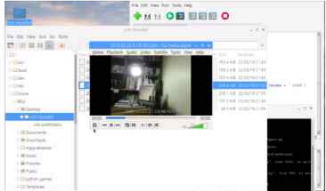
- 적외선 센서가 물체를 감지하면 컨베이어 벨트가 잠시 멈추었다가 돌아가는 과정(후에 카메라 인식을 위해 중간에 멈추도록 함)

	
<pre> void loop() { // put your main code here, to run repeatedly: int volt=map(analogRead(A0), 0, 1023, 0, 5000); distance=(27.61/(volt-0.1696))*1000; Serial.print(distance); Serial.println("cm"); Serial.println(""); delay(50); if (distance<15) { servo.write(110); delay(1000); servo.write(93); delay(3000); } else{ servo.write(110); //delay(3000); } } </pre> Windows 정품 인증	

3월 22일 ~ 3월 31 : 감굴 분류를 위해 선과기 회전 테스트

- 적외선 센서가 물체를 감지하면 기계가 돌아가서 물체를 떨어뜨리고 나서 다시 원래 상태로 돌아오는 과정

4월 5일 : 감굴을 촬영하기 위해 라즈베리파이 카메라 동작을 위한 자료 조사

https://sykiv66.github.io/blog/2020/10/31/picamera.html#4-%EC%82%AC%EC%A7%84-%EC%B0%8D%EB%8A%94-%ED%8C%8C%EC%9D%B4%EC%8D%AC-%EC%8D%94%EB%93%9C  [Raspberry pi] 카메라 모듈 제어하기 - sykiv66 blog sykiv66 blog Archive About raspberry pi python camera Oct 31, 2020 sykiv66.github.io	
https://m.blog.naver.com/entry.naver?categoryId=22158485417&referrerCode=1  라즈베리파이 CCTV - 1. 라즈베리파이로 사진, 동영상 촬영하... 라즈베리파이 CCTV 프로젝트 첫 단계입니다 이번에는 라즈베리파이로... m.blog.naver.com	

4월 12일 : 라즈베리파이 카메라가 찍은 사진을 파일로 저장하기

- 감굴 선과장치에 카메라를 설치해 감굴을 찍은 후 당도 및 결점을 찾기 위해 은성이에게 보내야하기 때문에 우선 사진 파일을 저장해야한다.
- 라즈베리파이에서 사진을 찍고 처리하는 코드가 대부분 파이썬으로 되어있었다.

https://ybwworld.tistory.com/116 t - Windows 메모장 서식(O) 보기(V) 파일(F) 편집(E) 형식(S) 도구(T) 도움말(H) 설정(O) 실행(E) 종료(X) [Python/파이썬] with ~ as ~ 구문, Magic ... 파이썬을 활용하다보면 파일을 열고 수정한 후, 닫는 ... ybwworld.tistory.com	
https://datascienceschool.net/01%20python/02.15%20%ED%8C%8C%EC%9D%B4%EC%8D%AC%EC%97%90%EC%84%9C%20%EB%82%AC%EC%A7%9C%EC%99%80%20%EC%88%9C%EA%B0%84%20%EB%88%A4%EB%83%A8%EA%B8%B0.html 2.15 파이썬에서 날짜와 시간 다루기 - 데이터 사이언스 스쿨 2.15 파이썬에서 날짜와 시간 다루기 2 날짜와 시간은 파이썬에서 기본으... datascienceschool.net	

4월 19일 : 라즈베리파이와 아두이노 연동

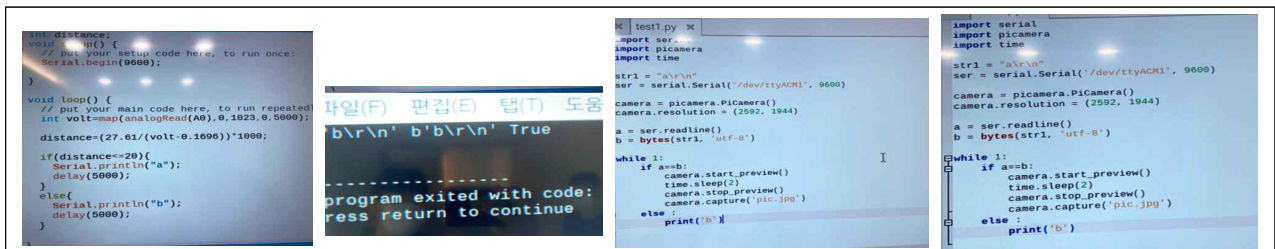
- 감굴 선과기의 움직이는 동작과 감굴이 감지되면 멈추는 동작은 아두이노로 만들었다.

- 감귤 선과기에 카메라 기능을 추가하려고하니 라즈베리파이 카메라를 사용해야해서 아두이노와 라즈베리파이를 같이 동작할 수 있도록 해야해서 라즈베리파이와 아두이노를 같이 사용하는 방법을 찾아보고 해봤다.



4월 26일 ~ 5월 10일 : 아두이노에서 읽은 값을 비교하여 카메라 동작 제어

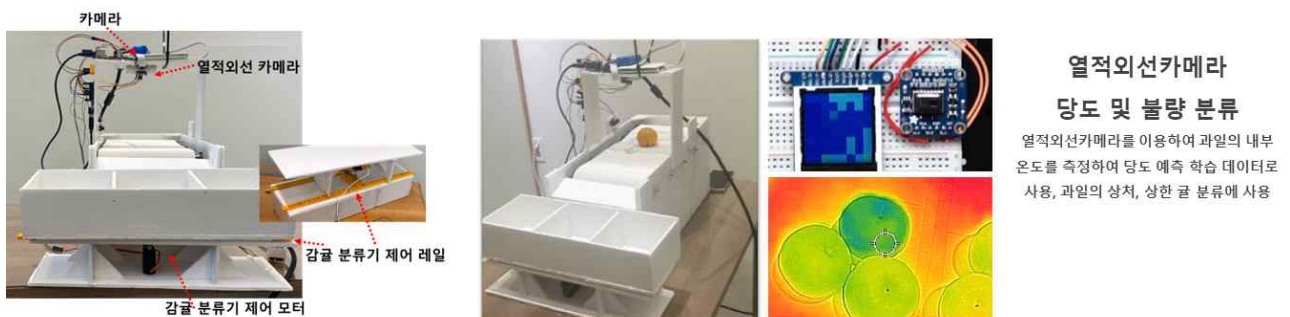
- 문제점 : 라즈베리파이에서 읽은 아두이노 시리얼 통신 값은 바이트로 b'a/r/n'이고 문자열 a는 바이트로 b'a'인데 아두이노에서 받은 값을 바이트에서 문자열로 디코딩 했을때는 두 값 모두 "a"이지만 바이트 값이 달라 비교했을때 거짓이 뜬다. 따라서 문자열 a "a/r/n"으로 지정해서 바이트 값으로 변환하면 b'a/r/n'이므로 이 상태에서 두 바이트 값을 같다고 컴퓨터가 인식하도록 하였다.



6월 ~ 7월 : 감귤 선과기 동작 실험



8월 ~ 9월 : 아두이노의 열적외선 카메라의 측정 값 라즈베리파이로 전송 실험



9월 ~ 10월 : 동작 테스트



[3] 감귤 분류를 위한 자료조사 및 프로그램 제작[담당자 : 장은성, 조왕현]

2월 1일 ~ 2월 15일 : YOLO(You Only Look Once) 선택 이유 및 자료조사

- 귤의 신선도 구분을 위하여 객체 탐지 모델 중에 옴로를 사용하게 되었다. 왜냐하면 사진에 무작위로 박스를 그리는 R-cnn보다는 사진을 그리드로 나누어서 구역당 일정한 양에 박스를 그려 물체를 감지하여 더 빠르기 때문이다.
- 옴로는 먼저 사진을 S*S 크기의 그리드로 나눈다. 그리고 그리드의 각 구역당 B개의 박스를 그린다. 박스 안에 물체가 있을 확률이 일정 확률을 넘지 않으면 박스를 삭제한다. 그 뒤 그려진 박스들 중 겹쳐진 박스들을 삭제한다. 하지만 일정 부분 이하로 겹쳐진 경우 그곳에 다른 물체가 존재할수있기 때문에 지우지 않는다. 그 뒤 남은 박스들에 클래스를 정해준다.

○ 2월 22일 ~ 3월 8일 : YOLO 설치

- YOLO는 이미지 기반 객체 검출 알고리즘으로 새로운 이미지가 입력되었을 때 학습된 가중치를 가지고 객체의 클래스를 예측하는 알고리즘이다. 옴로는 이미지 내의 바운딩 박스와 클래스 정확도를 예측 하여 이미지 내에 객체의 종류와 위치를 검출하여 보여준다.
- 옴로와 C에 대한 지식이 없어도 사전에 훈련되어 있는 옴로모델과 가중치를 제공해주어 사용자가 자신의 데이터를 준비한 후 모델의 파라미터를 커스터마이징 하여 옴로를 재학습시킬 수 있게 해 주는 프레임워크이다.
- YOLO 라벨링을 위한 프로그램 설치

```
Automatic yolo-labeler
https://github.com/abpanchal95/yolo-labeler

• abpanchal95가 만든 yolo-labeler는 자동으로 이미지에서 객체를 탐지해 Bounding Box 좌표 정보를 생성해준다.
  • 하나의 객체만 탐지할 수 있다.

설치

$ pip3 install yolo-labeler

사용

from yolo_labeler import yolo_labeler
yolo_labeler.run(args)

• 명령어를 실행한다.

$ python3 test.py --input_path (img_path or img_folder) --output_path (output_img_folder) --output_text_path (output_text_folder) --yolo_label (class_number) --png_path (png_folder)

Flag

• --input_path
  • 원본 이미지 경로 또는 이미지가 들어있는 폴더 경로
• --output_path
  • 탐출된 이미지가 담길 폴더 경로
  • 이미지 원본이 저장된다.
```

```
Darknet

• YOLO 학습을 위해 개발된 오픈소스 신경망 프레임워크

설치

• OpenCV 설치
  • 실시간 이미지 프로세싱에 종속된 온 라이브러리

$ sudo apt-get install libopencv-dev

• CUDA 설치
  • NVIDIA GPU에 가장 효율적 성능을 사용할 수 있도록 하는 API 모델
  • 그래픽카드와 버전을 맞춰야 한다.

$ sudo apt-get install cuda

• cuDNN 설치
  • CUDA의 딥러닝 라이브러리 (CUDA Deep Neural Network Library)
  • CUDA와 버전을 맞춰야 한다.

$ sudo sh cuda_10.0.130_linux_x86_64.sh
$ sudo sh cuda_10.0.130_linux_x86_64.sh

• vcpkg 설치 및 빌드
  • C/C++ 패키지 라이브러리를 간단하게 설치할 수 있는 패키지 관리자

$ git clone https://github.com/microsoft/vcpkg
$ cd vcpkg
$ vcpkg install opencv
$ vcpkg install pytorch

• Darknet 설치 및 빌드

$ git clone https://github.com/jdrowe/darknet
$ cd darknet
$ make
$ mv libdarknet.so /usr/lib/

• Darknet 실행

$ ./darknet.exe detector test cfg/coco.data cfg/yolov3.cfg yolov3.weights (img)
```

- 옴로 커스텀 학습을 하기 위해서는 커스텀 데이터 셋이 필요하다. 커스텀 데이터 셋에는 이미지, Bouding Box 좌표를 담은 파일이 담겨있다. Bouding Box 좌표는 사람이 직접 이미지에서 검출할 객체마다 하나하나 지정해줘야 한다.

- [출처] <https://github.com/abpanchal95/yolo-labeler>

- 설치 과정 중 발생한 문제

1. 객체를 탐지하기 위해 u2net 모델을 사용하는데 이 모델을 다운로드하는 과정에서 에러가 발생한다. `U2net.pth` 파일을 웹에서 직접 다운로드 받은 뒤 `C:\Users\{Username}\u2net\U2net.pth` 에 저장한다.

2. 객체를 `python torch` 라이브러리 버전이 맞지 않을 경우 그래픽카드를 찾을 수 없다는 에러가 발생한다. i. Python torch 버전과 CUDA 버전을 맞춰준다.

○ 3월 15일 : 감귤 당도 예측 방법에 대한 아이디어

- 사과와 감귤의 색깔을 이용한 당도 예측을 하는 자료를 보고 우리지역의 대표적인 농작물인 귤의 껍질색을 이용한 당도 예측이 가능한지 해보고 싶었다.
- 그래서 우리는 감귤의 사진을 찍어 감귤의 껍질이 가지는 색의 분포를 확인해 봤다.
- 감귤의 껍질의 색의 분포 데이터와 감귤 당도 데이터를 수집해서 껍질의 색 분포 데이터로 감귤의 당도를 분석할 수 있는지 실험하려고 한다.

○ 3월 15일 ~ 4월 5일 : 사진 속에 감귤이 있는지 찾기

- YOLOv5 모델 학습

- 모델 사용해서 감귤이 있는지 찾는



```
!python /content/drive/MyDrive/yolov5/train.py --img 640 --batch 7 --epochs 30 --data /content/drive/MyDrive/dataset/train_image/dataset/models/yolov5s.yaml --weights yolov5s.pt --name dataset
```

```
!python "C:/Users/sooks/workspace/yolov5/detect.py" --img-size 640 --conf 0.5 --source "C:/Users/sooks/workspace/images/test03.jpg"
```

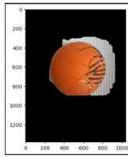
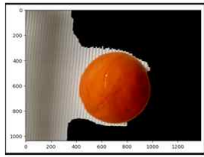
○ 4월 12일 ~ 4월 26일 : 사진 속 귤 확인 후 배경색 삭제



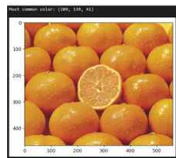
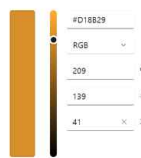
하드웨어에서 귤을 찍어서 소프트웨어로 사진을 넘긴 뒤 다음에 나올 과정을 거쳐서 당도가 높다면 컨베이어 벨트가 회전하여 바구니에 귤을 넣습니다.



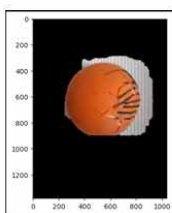
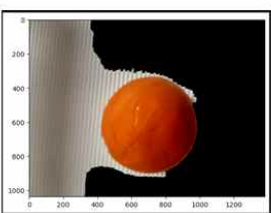
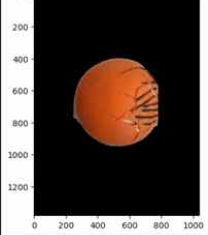

사진에 귤이 없이 잘못 찍혔을 수도 있으니 YOLO를 사용하여 귤이 찍혀 있나 한번 더 확인한 후에 다음 단계로 넘어가게 됩니다.

색 추출의 편의를 위해 배경을 제거한 뒤 색 추출을 하게 됩니다. 하지만 제거한 배경이 검은색으로 인식되어 귤의 색이 안 나오는 문제가 있었지만 검은색과 하얀색에 가까운 색을 제외하고 추출하게 하여 귤의 색을 추출합니다.

- YOLO사진을 넣으면 박스 쳐서 나옴, 그리고 실시간 웹 캡 인식이 가능하여 인식 가능
- grabcut 사용해서 배경 제거 : 좌표 지정이 피로함
- 색 추출->배경색 까지 인식 돼서 배경 삭제 ->근데 삭제된 배경 때문에 검은색 추출 돼서 검은색 흰색 제외하고 추출 ->배경 제거가 깔끔하게 안됨 -> YOLO에서 감귤의 좌표를 가져와서 정확도 향상

```
det[:, :4] = scale_boxes(im_shape[2:], det[:, :4], im_shape).round()

print("det : ", det)
positions = []
for *xyxy, conf, cls in reversed(det):
    positions.append([int(xyxy[0].item()), int(xyxy[1].item()), int(xyxy[2].item())-xyxy[0].item()])
print(positions)
print(type(int(xyxy[0].item()))))

if positions is not None:
    image_rgb = cv2.cvtColor(imgs, cv2.COLOR_BGR2RGB)
    rectangle = positions[0]

    mask = np.zeros(image_rgb.shape[:2], np.uint8)

    bgModel = np.zeros((1, 65), np.float64)
    fgModel = np.zeros((1, 65), np.float64)

    cv2.grabCut(image_rgb, mask, rectangle, bgModel, fgModel, 5, cv2.GC_INIT_WITH_RECT)
    mask_2 = np.where((mask==2) | (mask==0), 0, 1).astype('uint8')

    image_rgb_nobg = image_rgb * mask_2[:, :, np.newaxis]

    image = Image.fromarray(image_rgb_nobg)
```

- grabcut 사용해서 배경 제거 좌표 지정 필요함
- YOLO 모델에서 감귤 좌표 가져옴
- 정확도 향상
- 사진 속 감귤 껍질의 색 분포 알아내기
 - pillow(Python Image Library) 모듈 사용해서 가장 많은 색 3가지 추출
 - 제거된 배경(검은색)을 가장 많은 색으로 판단
 - >색 추출 시 검은색 제거
 - 회색 계열 제거(배경 제거가 제대로 안 됐을 수 있음)
 - 좌표를 가져와서 깔끔하게 사라지도록 하는 중

```

colors = image.getcolors(image.size[0]*image.size[1])
sorted_colors = sorted(colors, key=lambda x: x[0], reverse=True)

count=0
count1=0
for i in sorted_colors:
    r=i[1][0]
    g=i[1][1]
    b=i[1][2]
    average=(r+g+b)/3
    if abs(average-r)<=30 and abs(average-g)<=30 and abs(average-b)<=30:
        continue
    else:
        print(count+1,":", i[1])
        count+=1
        if count==3:
            break
cv2.imwrite(save_path, cv2.cvtColor(image_rgb_nobg, cv2.COLOR_RGB2BGR))

```

○ 5월 3일 : 사과를 이용한 껍질의 당도 분석 방법 조사 및 적용

선형회귀를 이용한 사과의 색상과 당도 분석

Analysis of Apple Colors and Sugar Contents Using Linear Regression

논문에서는 사과의 껍질 색과 측정된 당도의 관계를 분석하고자 한다.

사과는 경험적으로 껍질에 나타나는 색상과 내부의 당도가 연관이 있는 것으로 알려져 있다. 특히, 사과의 배 영역의 색상이 당도와 연관이 있음이 확인되었다.

-궤도 껍질의 색과 당도가 연관이 있을 것으로 예상

사과를 먹서기로 갈아서 즙으로 만들고, 즙을 통해 당도를 측정하였다.

-당도 측정기를 사용해 궤의 당도를 측정

그림 1은 사용된 사과의 배 부분의 사진이다. 배경 중앙의 색상은 당도와 무관하다고 판단하고 중앙 부분을 제외한 색상 정보만을 얻기 위하여 (a)에 편집 과정을 수행하여 (b)를 얻는다.(사과 영상 120개 사용)

-궤 사진의 배경과 궤 부분을 제거한 후 남은 부분에서 색을 추출

그림 2는 데이터 셋의 당도 분포를 보여주고 있다.

당도는 12-19 Brix 사이에 분포되어있으며, 당도의 분포가 15 근처인 사진이 30개로 가장 많다.

사과보다 붉을 것으로 예상

색상 공간 RGBhsvLab의 9개 색상과 당도의 상관계수를 알아보았다. 그림 8을 보면, v와 b는 당도에 +영향을 미치는 것으로 나타났다. v는 명도를 의미하므로, 흰색(255)에 가까울수록 당도도 높을 것 같아 나타날 가능성이 있다는 것을 알려주고 있다. 그리고 얼마나 붉은색에 가까운가를 나타내는 a는 당도와 관계가 약하게 나타났지만, 얼마나 노란색에 가까운가를 나타내는 b는 당도에 큰 영향을 주는 것으로 나타났다는 것을 알 수 있다.

h:색조 s:채도 v:명도 L:휘도(인간이 느끼는 밝기)
a:붉은색에 가까운 정도 b:노란색에 가까운 정도

1. 궤 사진 배경 제거
2. 그림 1의 (b)처럼 궤 꼭지 제거
3. 궤 색 추출(가장 많은 색 3가지)
4. 궤의 껍질 색과 당도의 관계 분석

○ 5월 4일 ~ 5월 10일 : 궤 꼭지 제거와 궤 색 추출(가장 많은 색 3가지 찾기)

- 궤의 색을 RGB로 구분하는 것으로는 궤 색의 분류가 부정확해서 추가로 hsv 값까지 추가해서 6개의 값으로 색 추출

궤 꼭지 제거 실패[문제점]

1. 꼭지의 색을 찾아서 제거
2. 꼭지를 찾는 모델 만들어서 제거

-꼭지만 제거하기 어려움

-모델 학습에 시간이 오래 걸림

-코드 전체를 수정해야 함

꼭지 부분이 상대적으로 적어 데이터에 큰 영향을 주지 않을 것으로 판단

궤 색 추출(가장 많은 색 3가지)

기준

-사진 한 장의 RGB 값만 출력

수정

-사진 여러 장의 hsv 값까지 출력

```

h,s,v=coloursys.rgb_to_hsv(r/255.,g/255.,b/255.)
h,s,v=int(h*360),int(s*100),int(v*100)
print(r,g,b,h,s,v, end=" ")

```

```

[[262, 396, 537, 569]]
<class 'int'>
1 : 179 69 18 19 89 70
2 : 180 70 19 19 89 70
3 : 193 74 18 19 90 75
[[259, 336, 565, 602]]
<class 'int'>
1 : 184 72 22 18 88 72
2 : 183 71 21 18 88 71
3 : 174 66 19 18 89 68
[[329, 625, 559, 614]]
<class 'int'>
1 : 164 48 23 10 85 64
2 : 163 47 22 10 86 63
3 : 165 49 24 10 85 64

```

○ 5월 11일 : 가상 데이터로 모델 학습을 위한 논의

- 10~20개의 데이터 생성
- 가상 데이터 생성(껍질 색, 당도)
- 감귤 데이터로 모델 학습 -> 실제 감귤 사진으로 데이터 생성, 학습 예정

○ 5월 12일~5월 15일 : 가상 데이터 만들기

- 감귤 사진에서 감귤의 색을 가져옴(grabcut, PIL)
- 당도는 측정할 수 없으므로 임의로 지정



감귤 데이터 생성

굴	1	2	3	당도(brix)
1	248 120 0 29 100 97	249 124 0 29 100 97	235 109 0 27 100 92	7.3
2	255 170 2 39 99 100	254 167 2 39 99 99	255 168 3 39 98 100	11.5
3	190 103 0 32 100 74	186 100 1 32 99 72	192 105 0 32 100 75	12.9
4	249 132 1 31 99 97	254 166 30 36 88 99	253 164 34 35 86 99	9.6
5	255 168 1 39 99 100	253 164 0 38 100 99	253 168 0 39 100 99	15.2
6	231 111 40 22 82 90	231 111 38 22 83 90	230 104 37 20 83 90	9.8

감귤 데이터 생성

굴	1	2	3	당도(brix)
7	190 103 0 32 100 74	186 100 1 32 99 72	192 105 0 32 100 75	11.5
8	255 168 1 39 99 100	253 164 0 38 100 99	253 168 0 39 100 99	12.7
9	248 120 0 29 100 97	249 124 0 29 100 97	235 109 0 27 100 92	9.8
10	255 172 50 35 80 100	255 175 50 36 80 100	255 160 50 32 80 100	13.1
11	255 182 0 42 100 100	253 183 0 43 100 99	254 179 0 42 100 99	8.7
12	225 148 30 36 86 88	223 146 30 36 86 87	202 115 0 34 100 79	10.9

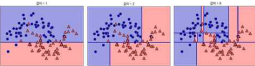
○ 5월 15일 ~ 5월 26일 : 가상 데이터로 모델 학습

- 의사 결정 나무 여러 개를 훈련시킨 후 결과를 종합해 예측
- 랜덤 포레스트 : 각 트리에서 변수를 무작위로 뽑아 여러 개의 트리를 만들어 결합
- 장점 : 결측치와 이상치가 많아도 정확도가 높음

의사 결정 나무(Decision tree)

데이터 사이에 존재하는 패턴을 예측 가능한 규칙들의 조합으로 나타낸다.

1. 데이터를 가장 잘 구분할 수 있는 질문을 기준으로 나눈다
2. 각 구역에서 1을 반복한다



랜덤 포레스트(Random Forest)

의사 결정 나무 여러 개를 훈련시킨 후 결과를 종합해(평균) 예측한다.

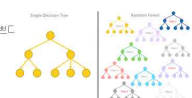
하나의 결정트리가 모든 변수(feature)를 사용하면 과대적합이 발생하므로 각 트리에서 변수를 무작위로 뽑아 여러 개의 트리를 만들고 결합한다.

장점: 각 변수의 중요성을 파악할 수 있다. 결측치와 이상치가 많아도 정확도가 높다. 과대적합이 잘 일어나지 않는다.

단점: 훈련 시 메모리 소모가 크다.

차원이 큰 데이터(예: 텍스트 데이터)에는 성능이 나쁘다

데이터의 수가 많으면 속도가 많이 떨어진다.



모델 학습

데이터 생성

-모델 학습을 위한 가상 데이터 생성(R,G,B,h,s,v,target)

```
x_train=np.array([
    [248, 120, 0, 29, 100, 97],
    [255, 170, 2, 39, 99, 100],
    [190, 103, 0, 32, 100, 74],
    [249, 132, 1, 31, 99, 97],
    [255, 168, 1, 39, 99, 100],
    [231, 111, 40, 22, 82, 90],
    [190, 103, 0, 32, 100, 74],
    [255, 168, 1, 39, 99, 100],
    [248, 120, 0, 29, 100, 97],
    [255, 172, 50, 35, 80, 100]])
y_train=np.array([0,1,1,0,1,0,1,1,0,1])
```

모델 학습

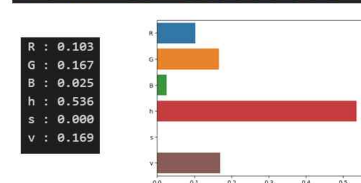
-결정트리의 수, 최대 깊이, 각 트리의 feature 수

```
clf=RandomForestClassifier(n_estimators=10,max_depth=4,random_state=0,max_features=2)
clf.fit(x_train,y_train)
```

모델 학습

모델 시각화

```
#feature의 importance 확인
for name, value in zip(['R','G','B','h','s','v'], clf.feature_importances_):
    print('({}) : ({}:{})'.format(name, value))
# feature importance column 정보 시각화 하기
plt.barplot(zip(clf.feature_importances_, y=['R','G','B','h','s','v']))
```



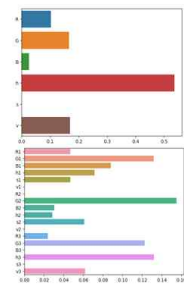
○ 5월 27일 : 실제 데이터로 학습 데이터 생성

가상 데이터로 모델 학습

이전 모델에서는 6개의 변수(R,G,B,h,s,v)를 사용

감귤에서 추출한 색은 가장 많은 3가지(6*3)

-> 3가지 색의 R,G,B,h,s,v 값을 모두 이어붙임



```
for name, value in zip(['R','G','B','h','s','v'], clf.feature_importances_):
    print('({}) : ({}:{})'.format(name, value))
plt.barplot(zip(clf.feature_importances_, y=['R','G','B','h','s','v']))
```

데이터 생성

1. 감귤 사진 촬영(test 0~8)
2. 편집 색 추출
3. 데이터 저장



```
0
[174, 73, 31, 17, 82, 68]
[175, 74, 32, 17, 81, 68]
[175, 72, 30, 17, 82, 67]

1
[137, 56, 27, 15, 80, 53]
[147, 58, 26, 15, 82, 57]
[156, 62, 26, 16, 83, 61]

2
[142, 60, 22, 19, 84, 55]
[139, 59, 22, 18, 84, 54]
[141, 59, 23, 19, 85, 55]

4
[150, 67, 27, 19, 82, 58]
[149, 66, 26, 19, 82, 58]
[148, 66, 28, 19, 81, 58]
```

	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100
0	174	73	31	17	82	68	175	74	32	17	81	68	175	72	30	17	82	67	156	62	26	16	83	61	142	60	22	19	84	55	139	59	22	18	84	54	141	59	23	19	85	55	150	67	27	19	82	58	149	66	26	19	82	58	148	66	28	19	81	58																																								

○ 6월 3일 : 당도 측정기를 이용하여 감귤의 당도를 측정하여 데이터 셋 생성

- 감귤 사진의 촬영 방법에 따른 그림자, 빛의 양에 따른 데이터 셋 비교
- 당도 측정기를 이용하여 감귤의 당도를 측정[감귤의 즙을 이용해 측정]
- 측정한 감귤의 당도가 모두 감귤의 평균 당도가 13~14브릭스보다 높았다. 그래서 당도가 낮은 감귤의 색은 임의로 설정하여 데이터 셋을 생성

감귤 사진 비교



감귤 사진 비교

-10~18은 그림자로 인해 감귤이 잘 인식되지 않음

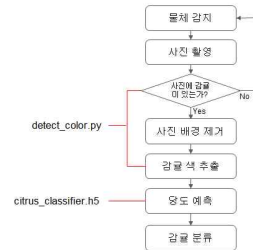
-20~28이 0~8에 비해 명도가 낮음

-색조, 채도는 큰 차이가 없음

27	7	8	-3	-2	11	31	9	10	-4	-3	12	27	7	6	-3	-2	10
-11	-10	-2	-3	0	-5	-2	-9	-4	-3	3	-1	11	-3	-4	-2	4	5
-2	-7	-3	-2	2	-1	-3	-5	-3	-2	2	-1	3	-5	-4	-1	4	5
6	-3	2	-3	0	2	2	-4	2	-3	-1	1	2	-3	5	-3	-3	1
19	9	2	1	1	7	18	6	4	-1	1	7	18	8	1	1	1	7
11	-3	-2	-3	2	4	13	-1	0	-3	2	5	10	-4	-3	-3	3	4
21	3	3	-3	0	8	24	6	6	-3	-2	9	24	6	6	-3	-1	10
22	6	4	-3	0	8	22	6	4	-3	0	9	-1	-12	-4	-5	2	5

감귤 당도 측정

1. 감귤의 껍질을 관 뒤 감귤의 즙을 짜 렘에 담는다.
2. 일정량의 즙을 당도측정기에 넣는다.
3. 당도를 측정한 뒤 앱을 사용해 값을 저장한다.
4. 측정한 당도를 색과 함께 저장한다.



○ 6월 10일 ~ 24일 : 상한귤(곰팡이), 덜익은 귤, 상처난 귤 분류

- 상한귤(곰팡이) : 파란색 계열, 대부분 밝은 색
- 덜익은 귤 : 초록색 계열, 빛 반사로 파란색 계열로 보임
- 오점이 있는 감귤 찾기 : 오점이 있는 귤에 해당하는 색이 있는지 확인
- 문제점 : 정확히 필요한 색만 포함되도록 범위를 설정하기 어려움, 빛 등의 영향으로 해당 색이 일부 포함되었을 수 있음

분류해야 하는 귤

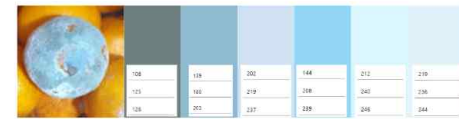
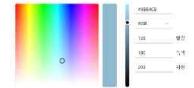
1. 상한 귤(곰팡이)
2. 덜 익은 귤
3. 상처난 귤



1. 상한 귤(곰팡이)

- 파란색 계열, 대부분 밝은 색, 첫 번째는 꼭지 색

R : 140~220 G : 180~240 B : 200~245



2. 덜 익은 귤

- 초록색 계열, 첫 번째는 빛 반사로 파란색 계열로 보임
- r, b 값 100 이하
- g 값 100 이상



오점이 있는 감귤 찾기

오점이 있는 귤에 해당하는 색이 있는지 확인

- 감귤의 색 모두 확인

-> 정확히 필요한 색만 포함되도록 범위를 설정하기 어려움(다른 색이 인식 될 수 있음). 빛 등의 영향으로 해당 색이 일부 포함되었을 수 있음

- 가장 많은 색 n가지 중 확인

-> 해당하는 색의 비율이 적을 수 있음



지금은 가장 많은 색 100가지 중에 해당하는 색이 있는지 확인함

- 가장 많은 색 최대 100가지를 확인해 오점이 있는지 확인

- 기존의 감귤 색 추출 코드에 추가

```

else:
    count+=1
    if count<=3:
        h,s,v=colors.rgb_to_hsv(r/255.,g/255.,b/255.)
        h,s,v=int(h*360),int(s*100),int(v*100)
        print(r,g,b,h,s,v,end="")
        str_color+=str(r)+" "+str(g)+" "+str(b)+" "+str(h)+" "+str(s)+" "+str(v)
        if r<140 and r<220 and g<180 and g<240 and b<200 and b<245:
            print("곰팡이",end=" ")
            break
        elif r<100 and g<100 and b<100:
            print("덜 익은",end=" ")
            break
        if count>=100:
            break
        str_color+=" "
    
```

- 곰팡이

173 81 32 20 81 67//166 85 33 20 82 72//175 63 32 21 81 68//곰팡이//180 182 205 235 12 80

- 덜 익은 귤

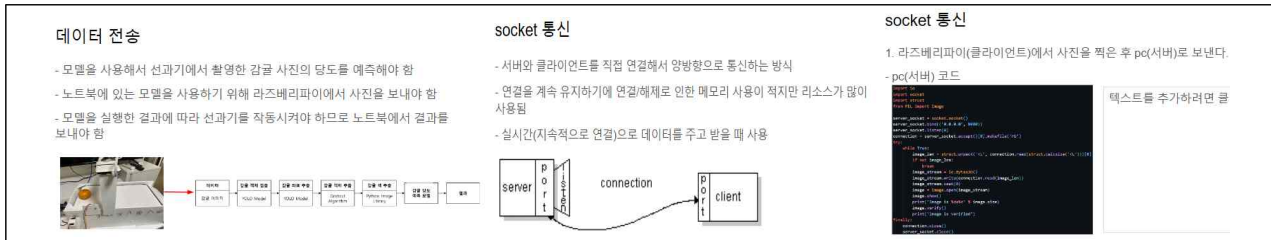
yolo 모델이 인식하지 못함



○ 6월 ~ 7월 : 감귤 선과기 전체 동작

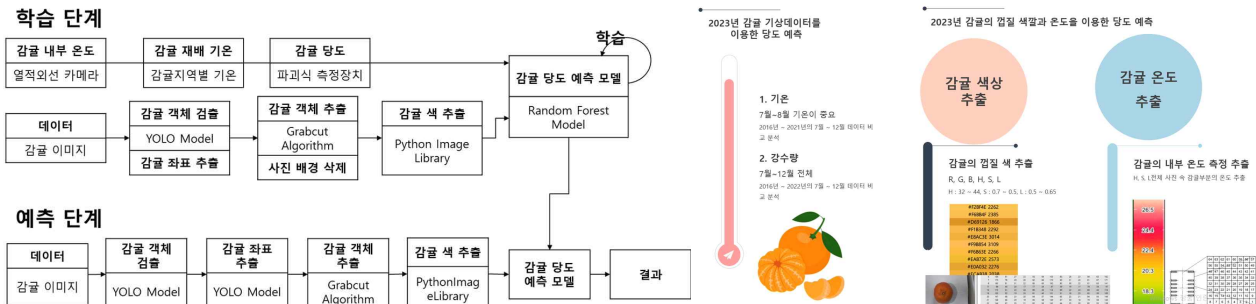
○ 7월 ~ 8월 : 귤 껍질 사진에서 가장 많은 색 3가지 색을 추출

- 문제 해결 1 : 감귤의 배경을 제거하기 위해 검은색, 흰색을 제외하고 추출하였으나 배경제거가 잘 되지 않았음, YOLO 감귤 좌표값을 이용해정확도향상
- 문제 해결 2 : 귤 꼭지 제거와 귤 색 추출(가장 많은 색 3가지 찾기) -> 귤의 색을 RGB로 구분



하여 분류할 경우 정확도가 높지 않았다. 그래서 추가적으로 hsv 값 추가 6가지 값의 색 추출 하였음, 귤 꼭지 제거의 경우 정확도가 높지 않아 가장 많이 분포하는 3가지 색을 찾으려 함

- 문제 해결 3 : 감귤 껍질 색을 이용한 분류의 정확도를 높이기 위해 기상데이터, 열적외선 카메라의 감귤 온도 내부값 등을 사용함
- 8월 ~ 9월 : 감귤 분류 실험 : 학습데이터
 - 1차 학습 데이터 : 감귤 사진 속 가장 많이 검출되는 3가지 색을 추출(R,G,B,H,S,V), 감귤철이 아니라 인터넷에서 감귤 사진을 캡처해서 실험
 - 2차 학습 데이터 : 라즈베리카메라가 촬영한 30개의 감귤 사진과 당도 측정기로 측정한 당도값을 이용해 학습데이터 생성
 - 3차 학습 데이터 : 동일한 조명 속에서 감귤 120개를 촬영, 120개의 감귤 내부 온도 측정, 감귤 재배지의 기온 온도, 당도 측정기로 측정한 당도값 학습데이터
- 9월 ~ 10월 : 감귤 분류 실험



IV 출처

1. 비파괴 선별기 원리 : <https://m.cafe.daum.net/nirspectra/74LM/9>
2. 맛있는 과일을 알아볼 수 있는 당도측정기 원리 : <https://blog.naver.com/david293/222566291662>
3. 기존 수집된 감귤 당도 값과 기상 데이터(평균 온도, 습도, 강수량, 일사량, 평균 풍속)을 기반으로 측정되지 않은 날짜의 감귤 당도를 예측하는 AI 모델 : <https://koreascience.kr/article/JAKO202119759301791.pdf>
4. 국내 최초 인공지능 과일선과기 개발 화제 : <http://kenews.co.kr/mobile/article.html?no=82079>
5. 근적외 분광분석법을 응용한 사과의 비파괴 품질 측정 가능성 조사 : <https://scienceon.kisti.re.kr/commons/util/originalView.do?cn=JAKO199803043008830&oCn=JAKO199803043008830&dbt=JAKO&journal=NJOU00034641>
6. 열화상카메라의 원리 : <https://brunch.co.kr/@seouledu/338>
7. 열화상카메라는 어떤 원리로 작동할까 : <https://www.flirkorea.com/discover/rd-science/how-do-thermal-cameras-work/>
8. 열화상카메라의 원리 : <https://smorning.tistory.com/112>
9. 분광광도계(Spectrometer)란? : <http://www.jinoptic.com/spectrometer/>
10. 배 수확후 관리-비파괴 선별기 : <https://m.blog.naver.com/gib6560/80124861555>
11. 분광광도법 : <https://stachemi.tistory.com/129>
12. 카메라 모듈 제어하기 : <https://syki66.github.io/blog/2020/10/31/picamera.html#4-%EC%82%AC%EC%A7%84-%EC%B0%8D%EB%8A%94-%ED%8C%8C%EC%9D%B4%EC%8D%AC-%EC%BD%94%EB%93%9C>
13. 라즈베리파이 카메라 제어하기 : <https://m.blog.naver.com/einsbon/221215848541?referrerCode=1>
14. 파이썬에서 날짜와 시간 다루기 : <https://datascienceschool.net/01%20python/02.15%20%ED%8C%8C%EC%9D%B4%EC%8D%AC%EC%97%90%EC%84%9C%20%EB%82%A0%EC%A7%9C%EC%99%80%20%EC%8B%9C%EA%B0%84%20%EB%8B%A4%EB%A3%A8%EA%B8%B0.html>
15. 아두이노 라즈베리파이 연동 : <http://www.3demp.com/community/boardDetails.php?cbID=232>
16. 라즈베리파이 RPi.GPIO 모듈 : <https://m.blog.naver.com/pk3152/221368513358>
17. 라즈베리파이 GPIO로 LED 제어하기 : <https://m.blog.naver.com/PostView.naver?isHttpsRedirect=true&blogId=pk3152&logNo=221368477491>
18. 아두이노와 라즈베리파이 시리얼 통신하기 : <http://www.notforme.kr/archives/756>
19. 열화상 카메라 : <https://www.adafruit.com/product/3538>
20. YOLO: <https://blog.naver.com/PostView.nhn?isHttpsRedirect=true&blogId=sogangori&logNo=220993971883&parentCategoryNo=6&categoryNo=&viewDate=&isShowPopularPosts=true&from=search>
21. YOLO 구현:
 - [https://panggu15.github.io/detection/%EA%B0%84%EB%8B%A8%ED%95%9C-YOLO-%EA%B5%AC%ED%98%84\(OpenCV\)/](https://panggu15.github.io/detection/%EA%B0%84%EB%8B%A8%ED%95%9C-YOLO-%EA%B5%AC%ED%98%84(OpenCV)/)
 - <https://bong-sik.tistory.com/16>
 - <https://velog.io/@kimsoohyun/YOLO-%EC%9D%B4%EB%AF%B8%EC%A7%80-%EB%9D%BC%EB%B2%A8%EB%A7%81%EC%9D%84-%EC%9C%84%ED%95%9C-labelImg-%EC%82%AC%EC%9A%A9%EB%B2%95>
 - <https://developeryoung.tistory.com/15>

- 22. 배경제거(grabcut): <http://www.gisdeveloper.co.kr/?p=6747>
- 23. 색 추출(PIL): <https://conansjh20.tistory.com/61>
- 24. 사과와 당도 분석 논문:
<https://scienceon.kisti.re.kr/commons/util/originalView.do?cn=JAKO202208148789847&dbt=JAKO&koi=KISTI1.1003%2FJNLJAKO202208148789847>
- 25. 랜덤포레스트:<https://zephyrus1111.tistory.com/249#c3>
- 26. 랜덤포레스트 구현:
 - <https://for-my-wealthy-life.tistory.com/15>
 - <https://brunch.co.kr/@parkkyunga/95>
- 27. socket
 - <https://kadosholy.tistory.com/125>
 - <https://www.daleseo.com/what-is-a-socket/>
- 28. socket 예제
 - <https://m.blog.naver.com/PostView.naver?isHttpsRedirect=true&blogId=zeta0807&logNo=222144886241>
 - <https://jayharvey.tistory.com/6>